ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH

ĐỒ ÁN TỔNG HỢP CÔNG NGHỆ PHẦN MỀM

Đề tài

Thiết kế và phát triển game 2D bằng Unity

Giảng viên hướng dẫn:	Nguyễn Minh Tâm	
Nhóm:	05	
Sinh viên thực hiện:	Lưu Ngọc Huyền	2352418
	Huỳnh Nguyễn Nhật Huy	2352380
	Dương Thị Bảo Trân	2353205
	Bùi Võ Bảo Ngọc	2352808
	Bùi Phạm Ngọc Khôi	2352599

HO CHI MINH CITY, DECEMBER 2025

Contents

1 Requirement elicitation - Tìm hiểu yêu cầu **4**
1.1 Xác định động cơ của dự án. Dự án mong đợi thực hiện những gì? Phạm vi của dự án là gì? . . . 4
1.1.1 Xác định động cơ của dự án . . . 4
1.1.2 Dự án mong đợi thực hiện những gì . . . 4
1.1.3 Phạm vi của dự án . . . 5
1.2 Phân tích những sản phẩm hiện có (related works) trên thị trường. Mô tả tất cả các yêu cầu chức năng và phi chức năng của hệ thống mong muốn. Vẽ sơ đồ use-case cho toàn bộ hệ thống. . . . 6
1.2.1 Phân tích những sản phẩm hiện có trên thị trường và rút ra kết luận 6
1.2.2 Yêu cầu chức năng và phi chức năng . . . 8
1.2.3 Vẽ use-case diagram cho toàn bộ hệ thống . . . 9

2 Mô hình hóa hệ thống **18**
2.1 Activity Diagrams . . . 18
2.2 Sequence Diagrams . . . 24
2.3 Class Diagram . . . 34
2.3.1 Mô tả các phương thức trong Class Diagram . . . 34

3 Thiết kế kiến trúc **39**
3.1 ERD và Relational Schema . . . 39
3.1.1 ER Diagram . . . 39
3.1.2 Relational Schema . . . 40
3.2 Phương pháp tiếp cận kiến trúc để triển khai hệ thống . . . 43
3.2.1 Tổng quan kiến trúc . . . 43
3.2.2 Data Layer — Mapping ERD → Unity C# . . . 44
3.2.3 Mapping Relationship . . . 45
3.2.4 Logic Layer — Gameplay Managers . . . 45
3.2.5 Presentation Layer — UI . . . 46
3.2.6 Tích hợp — Cách hệ thống vận hành . . . 46
3.2.7 Lý do chọn kiến trúc này . . . 47

1 Requirement elicitation - Tìm hiểu yêu cầu

1.1 Xác định động cơ của dự án. Dự án mong đợi thực hiện những gì? Phạm vi của dự án là gì?

1.1.1 Xác định động cơ của dự án

Trong bối cảnh ngành công nghiệp game ngày càng phát triển, nhu cầu trải nghiệm những tựa game giả lập có chiều sâu và khả năng tương tác cao ngày càng tăng. Tuy nhiên, đa phần các trò chơi giả lập hiện có thường chỉ tập trung vào cơ chế sản xuất tuyến tính, thiếu sự gắn kết giữa người chơi, thế giới trong game, và cốt truyện.

Dự án này được thực hiện nhằm giải quyết khoảng trống đó bằng cách phát triển một trò chơi giả lập rèn đồ 2D phong cách pixel, lấy cảm hứng từ các tựa game như Graveyard Keeper và Papers, Please. Trong game, người chơi sẽ quản lý một tiệm rèn, chế tạo vũ khí, giao đồ theo yêu cầu, đồng thời tương tác với các NPC và ảnh hưởng đến diễn biến của thế giới trong game.

Ngoài việc tạo ra một sản phẩm có giá trị trải nghiệm, dự án này còn là cơ hội để nhóm thực hành toàn bộ quy trình phát triển phần mềm: từ phân tích yêu cầu, thiết kế hệ thống, xây dựng kiến trúc, triển khai MVP, cho đến kiểm thử và báo cáo kết quả. Đây là cơ sở để rèn luyện kỹ năng lập trình, làm việc nhóm, và tư duy thiết kế hệ thống.

1.1.2 Dự án mong đợi thực hiện những gì

Dự án **Soul in Forge** được kỳ vọng sẽ xây dựng một trò chơi giả lập rèn đồ 2D pixel, tập trung vào việc mô phỏng chân thực các công đoạn rèn, quản lý tài nguyên và hoàn thành các nhiệm vụ được giao bởi NPC. Trong phạm vi đồ án, dự án mong đợi đạt được:

- Hoàn thiện gameplay lõi: Người chơi trải qua chu trình lặp lại trong mỗi nhiệm vụ gồm: nhận quest từ Taskboard, thu thập hoặc mua nguyên liệu, nung quặng trong lò, rèn kim loại, gia công gỗ, mài vũ khí, và cuối cùng giao sản phẩm để nhận phần thưởng.

- Phát triển hệ thống trạm chế tác (Crafting Stations): Lò nung, đe rèn, bàn thợ mộc. Mỗi trạm gắn liền với một mini-game thao tác riêng, giúp tăng tính đa dạng và thử thách.

- Cơ chế nhiệm vụ và phần thưởng: Các quest có giới hạn thời gian, khi hoàn thành sẽ mang lại vàng và uy tín (Reputation). Nếu thất bại, người chơi bị giảm uy tín,

dẫn đến ảnh hưởng trực tiếp tới độ tin cậy và số lượng quest trong các ngày tiếp theo.

- Hệ thống Rank: Uy tín tích lũy sẽ giúp người chơi lên Rank mới. Rank càng cao thì càng mở khóa nguyên liệu và công thức vũ khí cao cấp hơn.
- Không gian game cơ bản: Bao gồm ba khu vực chính – tiệm rèn (Forge Scene), cửa hàng nguyên liệu (OreShop), và khu phố (Street Scene) làm cầu nối, tạo sự liền mạch trong trải nghiệm.

1.1.3 Phạm vi của dự án

Dự án **Soul in Forge** được thiết kế để triển khai ban đầu trên nền tảng PC (Windows), sử dụng engine Unity với đồ họa pixel 2D góc nhìn top–down. Việc chọn nền tảng này nhằm đảm bảo tính ổn định trong phát triển, dễ dàng kiểm thử và phù hợp với yêu cầu của đồ án môn học.

Trong phạm vi hiện tại, trò chơi tập trung vào xây dựng vòng lặp gameplay cơ bản của một tiệm rèn: nhận nhiệm vụ từ bảng quest, mua nguyên liệu tại cửa hàng, tiến hành các công đoạn chế tác (nung, rèn, mộc, mài), và giao sản phẩm để nhận phần thưởng. Người chơi có thể di chuyển trong ba khu vực chính gồm: Forge Scene (tiệm rèn), OreShop Scene (cửa hàng nguyên liệu) và Street Scene (khu phố kết nối giữa hai địa điểm).

Đối tượng hướng tới chủ yếu là người chơi đơn (single player) ở mức cơ bản, những người yêu thích thể loại game giả lập/mô phỏng, đặc biệt là các tựa game tương tự như Stardew Valley hoặc Graveyard Keeper. Do đó, trong khuôn khổ đồ án, dự án tập trung phát triển các tính năng gameplay cốt lõi thay vì mở rộng sang yếu tố cốt truyện phức tạp hay thế giới mở.

Về nội dung, phạm vi của phiên bản này chỉ bao gồm một số loại vũ khí cơ bản (kiếm, cung, khiên, rìu chiến) và nguyên liệu phổ biến (sắt, vàng, gỗ). Các yếu tố nâng cao như hệ thống NPC đa tuyến, cốt truyện phân nhánh, hay khu vực mở rộng (mỏ, rừng, lâu đài) sẽ được xem xét bổ sung ở các giai đoạn phát triển sau.

1.2 Phân tích những sản phẩm hiện có (related works) trên thị trường. Mô tả tất cả các yêu cầu chức năng và phi chức năng của hệ thống mong muốn. Vẽ sơ đồ use-case cho toàn bộ hệ thống.

1.2.1 Phân tích những sản phẩm hiện có trên thị trường và rút ra kết luận

1. Stardew Valley

Ưu điểm:

- Gameplay đa dạng: cho phép trồng trọt, chăn nuôi, khai thác, câu cá, chiến đấu và xây dựng mối quan hệ với NPC.
- Phong cách pixel art 2D nhẹ nhàng, gần gũi, dễ tiếp cận với đông đảo người chơi.
- Hệ thống phát triển nhân vật theo kỹ năng (farming, mining, fishing, combat, foraging) giúp tạo động lực lâu dài.
- Cộng đồng rộng lớn, hỗ trợ mod phong phú, tăng khả năng mở rộng.

Nhược điểm:

- Nội dung rộng nhưng thiếu tập trung, nhiều hoạt động dễ gây cảm giác "grind" và lặp lại.
- Các cơ chế chiến đấu, thám hiểm còn đơn giản, chưa cân bằng với các hoạt động nông trại.
- Thời lượng game dài, dễ gây quá tải với người chơi mới.

2. Papers, Please

Ưu điểm:

- Ý tưởng độc đáo: biến công việc kiểm soát nhập cảnh thành gameplay hấp dẫn, gây căng thẳng.
- Tính nhập vai cao, mỗi quyết định của người chơi ảnh hưởng đến cốt truyện, NPC và cả gia đình.
- Khai thác thành công các chủ đề đạo đức và trách nhiệm xã hội trong bối cảnh chính trị giả tưởng.

Nhược điểm:

- Gameplay chủ yếu xoay quanh kiểm tra giấy tờ, dễ tạo cảm giác lặp lại.
- Đồ họa đơn giản, thiếu sức hút với người chơi ưa thị giác.
- Không có yếu tố mở rộng thế giới hoặc nhiều tuyến nhân vật sâu.

3. Graveyard Keeper

Ưu điểm:

- Lối chơi mô phỏng kết hợp quản lý tài nguyên và crafting nhiều công đoạn, tạo chiều sâu gameplay.
- Chủ đề độc đáo, pha chút u ám và châm biếm, tạo sự khác biệt so với các game pixel thông thường.
- Hệ thống quest đa dạng, kết hợp yếu tố phiêu lưu (exploration, dungeon) và chế tạo (alchemy, crafting).

Nhược điểm:

- Nhịp game chậm, cơ chế quản lý tài nguyên phức tạp, dễ gây nản với người mới.
- Tuyến nhiệm vụ khó theo dõi, dễ khiến người chơi bị rối khi thực hiện nhiều quest song song.
- Hệ thống crafting đôi khi quá rườm rà, khiến tiến độ game bị kéo dài không cần thiết.

Kết luận: Dựa trên các ưu điểm và hạn chế quan sát được, **Soul in Forge** sẽ kế thừa phong cách pixel art và hệ thống phát triển tuyến tính từ Stardew Valley, học hỏi cảm giác trách nhiệm và tác động trực tiếp từ Papers, Please, cùng với cơ chế crafting nhiều công đoạn từ Graveyard Keeper. Đồng thời, dự án định hướng khắc phục hạn chế bằng cách giữ phạm vi hẹp nhưng tập trung, và thiết kế mini-game đa dạng để vừa tạo nét riêng vừa giảm cảm giác lặp lại.

1.2.2 Yêu cầu chức năng và phi chức năng

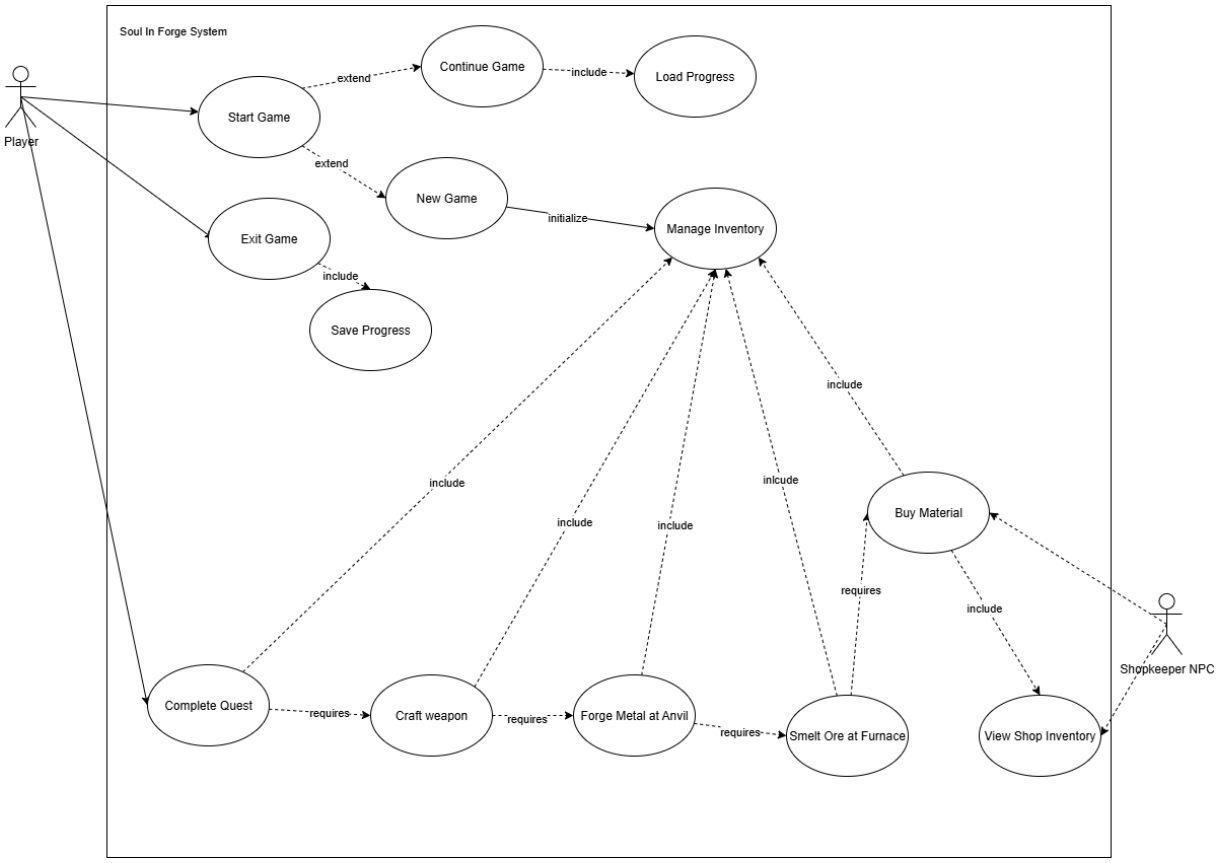
A. Yêu cầu chức năng

1. Người chơi (Player)
   - Nhận/huỷ/hoàn thành quest từ Taskboard/NPC.
   - Quản lý tài nguyên & kho đồ (mua ở OreShop, kiểm tra Inventory).
   - Chế tác theo công đoạn: Nung → Rèn → Mộc (mini-game thao tác).
   - Hệ thống Reputation/Rank (tăng/giảm uy tín, mở khoá công thức/nguyên liệu).
   - Nhận thưởng (vàng, mở khoá, vật phẩm).
   - Lưu/Tải tiến trình cục bộ (Save/Load offline).
2. Nhà thiết kế nội dung (Developer Tooling – nội bộ)
   - Trình cấu hình quest/công thức/nguyên liệu qua file JSON/ScriptableObject.
   - Bật/tắt, cân bằng tham số mini-game (thời gian, ngưỡng nhịp, độ khó).
   - Ghi log/bộ đo (completion rate quest, thời gian mỗi công đoạn) để tinh chỉnh.

B. Yêu cầu phi chức năng

- Hiệu năng: 60 FPS ổn định trên PC tầm trung; chuyển scene < 5s; input mini-game phản hồi tức thì.
- Khả dụng: Chạy offline hoàn toàn; không phụ thuộc mạng.
- Thân thiện: HUD rõ ràng (deadline quest, vàng, Reputation); tutorial ngắn cho mỗi mini-game.
- Mở rộng: Dễ thêm vũ khí/quest/công thức mới qua dữ liệu, không sửa code lõi.
- Bảo trì: Mã tách lớp (UI/Logic/Data); ScriptableObject cho dữ liệu gameplay; log lỗi tập trung.
- Tính di động: Lưu trữ save file cục bộ.

1.2.3 Vẽ use-case diagram cho toàn bộ hệ thống



Use Case chính (Primary Use Cases)

UC_StartGame: Bắt đầu game

Trường	Mô tả
ID Use Case	UC_StartGame
Tên Use Case	Bắt đầu Game
Tác nhân chính	Người chơi
Mô tả	Người chơi khởi động game và tải dữ liệu đã lưu hoặc bắt đầu với dữ liệu mặc định
Điều kiện tiên quyết	Người chơi đang ở Main Menu hoặc vừa mở game
Luồng chính	1. Người chơi nhấn nút "Start Game" 2. Hệ thống kiểm tra xem có save file hay không 3. **Nếu có save file:** - Hệ thống load dữ liệu đã lưu (inventory, vàng, quest, vị trí) - Chuyển người chơi đến scene đã lưu với spawn point tương ứng 4. **Nếu không có save file:** - Hệ thống khởi tạo dữ liệu mặc định: + Inventory trống + Vàng = 100 + Quest Log trống - Chuyển người chơi vào scene khởi đầu (ví dụ: Village Scene) tại spawn point mặc định
Luồng thay thế	**3a. Save file bị lỗi/corrupt:** 3a.1. Hệ thống hiển thị thông báo "Save file bị lỗi" 3a.2. Xóa save file cũ 3a.3. Quay lại bước 4 (khởi tạo dữ liệu mới)
Kết quả	Người chơi vào game và có thể bắt đầu chơi
Điều kiện sau	- Người chơi được spawn vào scene - Game sẵn sàng auto-save khi có thay đổi

UC_CompleteQuest: Hoàn thành nhiệm vụ

Trường	Mô tả
ID Use Case	UC_CompleteQuest
Tên Use Case	Hoàn thành nhiệm vụ
Tác nhân chính	Người chơi
Tác nhân phụ	TaskBoard, QuestSystem
Mô tả	Người chơi tương tác với TaskBoard để hoàn thành hoặc bỏ qua nhiệm vụ
Điều kiện tiên quyết	Người chơi đang ở gần TaskBoard
Luồng chính	1. Người chơi tiếp cận Task Board 2. Hệ thống hiển thị danh sách nhiệm vụ 3. Người chơi chọn 1 nhiệm vụ 4. Hệ thống kiểm tra điều kiện hoàn thành 5. **Nếu có đủ vật phẩm:** - Xác nhận hoàn thành - Nhận tiền - Tạo nhiệm vụ mới 6. **Nếu không đủ:** - Hiển thị yêu cầu chưa đủ 7. Người chơi chuyển sang làm "Buy Material" hoặc tiếp tục
Luồng thay thế	**Alt: Người chơi chọn bỏ qua nhiệm vụ** - Không có thay đổi về vật phẩm/tiền - Không có quest nào hoàn thành được - Di chuyển sang flow "Buy Material"
Kết quả	- Quest hoàn thành: Nhận tiền, tạo quest mới - Quest không đủ điều kiện: Chuyển sang thu thập nguyên liệu

UC_Buy: Mua nguyên liệu (Buy Materials)

ID Use Case	UC_Buy
Tên Use Case	Mua nguyên liệu
Tác nhân chính	Người chơi
Tác nhân phụ	NPC Cửa hàng nguyên liệu
Mô tả	Người chơi mua nguyên liệu thô cần cho chế tác tại cửa hàng nguyên liệu
Điều kiện tiên quyết	Người chơi có đủ vàng và chỗ trống trong kho
Luồng chính	1. Người chơi tiếp cận thương nhân 2. Hệ thống hiển thị nguyên liệu và giá bán 3. Người chơi chọn nguyên liệu 4. Hệ thống tính tổng chi phí 5. Người chơi xác nhận mua 6. Vàng bị trừ, nguyên liệu được thêm vào kho
Luồng thay thế	- Không đủ vàng - Không đủ chỗ trống - Hết hàng
Hậu điều kiện	Nguyên liệu có trong túi đồ, vàng giảm
Ngoại lệ	Giao dịch thất bại

UC_Smelt: Nung quặng tại lò (Smelt Ore at Furnace)

ID Use Case	UC_Smelt
Tên Use Case	Nung quặng tại lò
Tác nhân chính	Người chơi
Mô tả	Người chơi chuyển đổi quặng thô thành thỏi kim loại bằng lò luyện
Điều kiện tiên quyết	Người chơi có quặng trong kho và có quyền dùng lò
Luồng chính	1. Người chơi tương tác với lò 2. Hệ thống hiển thị giao diện nung 3. Người chơi chọn loại quặng 4. Thỏi kim loại được tạo ra
Luồng thay thế	- Các loại quặng yêu cầu kỹ thuật khác nhau
Hậu điều kiện	Quặng được chuyển thành thỏi kim loại trong kho
Ngoại lệ	Thiếu nhiên liệu, thất bại, hết thời gian
Bao gồm	UC_Inventory (Quản lý kho & vàng)

UC_Forge: Rèn kim loại tại đe (Forge Metal at Anvil)

ID Use Case	UC_Forge
Tên Use Case	Rèn kim loại tại đe
Tác nhân chính	Người chơi
Mô tả	Người chơi tạo hình thỏi kim loại thành linh kiện vũ khí tại đe
Điều kiện tiên quyết	Người chơi có thỏi kim loại và quyền dùng đe
Luồng chính	1. Người chơi tương tác với đe 2. Hệ thống hiển thị giao diện rèn 3. Người chơi chọn loại kim loại và bộ phận vũ khí 4. Tạo thành linh kiện vũ khí
Luồng thay thế	- Kim loại khác nhau có tính chất khác nhau
Hậu điều kiện	Linh kiện vũ khí được thêm vào kho
Ngoại lệ	Rèn thất bại, hao phí nguyên liệu, giới hạn thời gian
Bao gồm	UC_Inventory (Quản lý kho & vàng)
Mở rộng bởi	UC_Upgrade, UC_Repair

UC_Crafting: Chế tạo vũ khí (Crafting Weapon)

ID Use Case	UC_Crafting
Tên Use Case	Chế tạo vũ khí
Tác nhân chính	Người chơi
Tác nhân phụ	Bàn chế tạo (Crafting Table)
Mô tả	Người chơi sử dụng bàn chế tạo để tạo ra một loại vũ khí mới từ các nguyên liệu có sẵn.
Điều kiện tiên quyết	- Người chơi có đủ nguyên liệu cần thiết.
Luồng chính	1. Người chơi tiếp cận bàn chế tạo. 2. Hệ thống hiển thị giao diện chế tạo (Crafting UI). 3. Người chơi đặt nguyên liệu vào các ô tương ứng. 4. Hệ thống kiểm tra tính hợp lệ của công thức. 5. Nếu công thức đúng, hệ thống tạo vũ khí tương ứng và đưa vào túi đồ của người chơi.
Luồng thay thế	- Nguyên liệu đặt sai vị trí. - Nguyên liệu không đúng loại yêu cầu. - Túi đồ người chơi không còn ô trống. - Người chơi có thể thoát giao diện bất cứ lúc nào (hủy thao tác).
Hậu điều kiện	- Nếu thành công: vũ khí mới xuất hiện trong túi đồ người chơi. - Nếu thất bại: không có vật phẩm được tạo ra.
Ngoại lệ	- Hệ thống lỗi, không thể kết nối đến Crafting Table - Dữ liệu công thức chế tạo bị lỗi - Túi đồ đầy, không thể nhận vũ khí mới

Mô tả Tác nhân

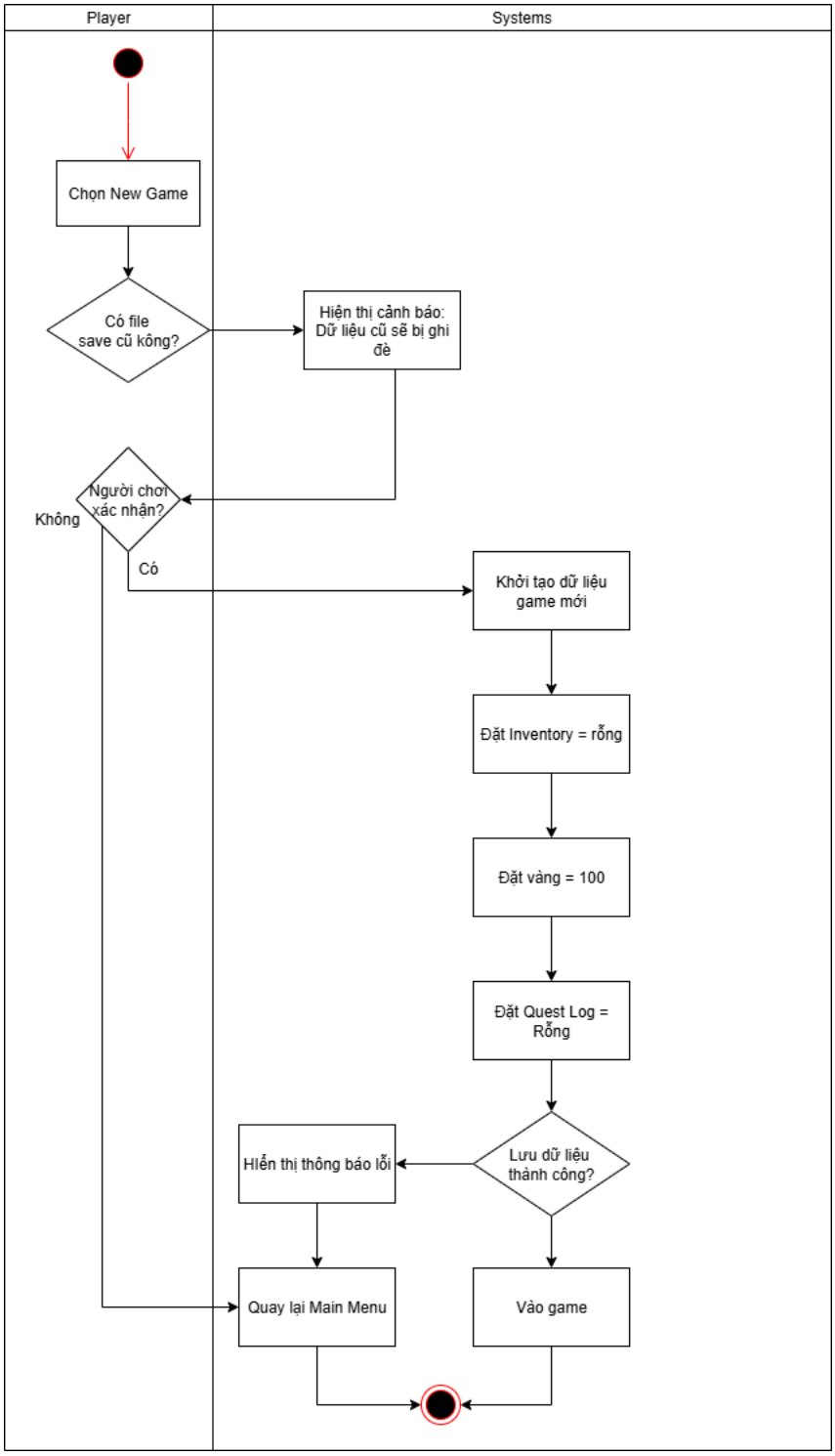
- **Người chơi (Player):** Người dùng chính tham gia rèn, nhận nhiệm vụ, quản lý tài nguyên và tiến trình.
- **NPC Cửa hàng nguyên liệu (Merchant):** Bán nguyên liệu và vật tư cho người

chơi.

2 Mô hình hóa hệ thống

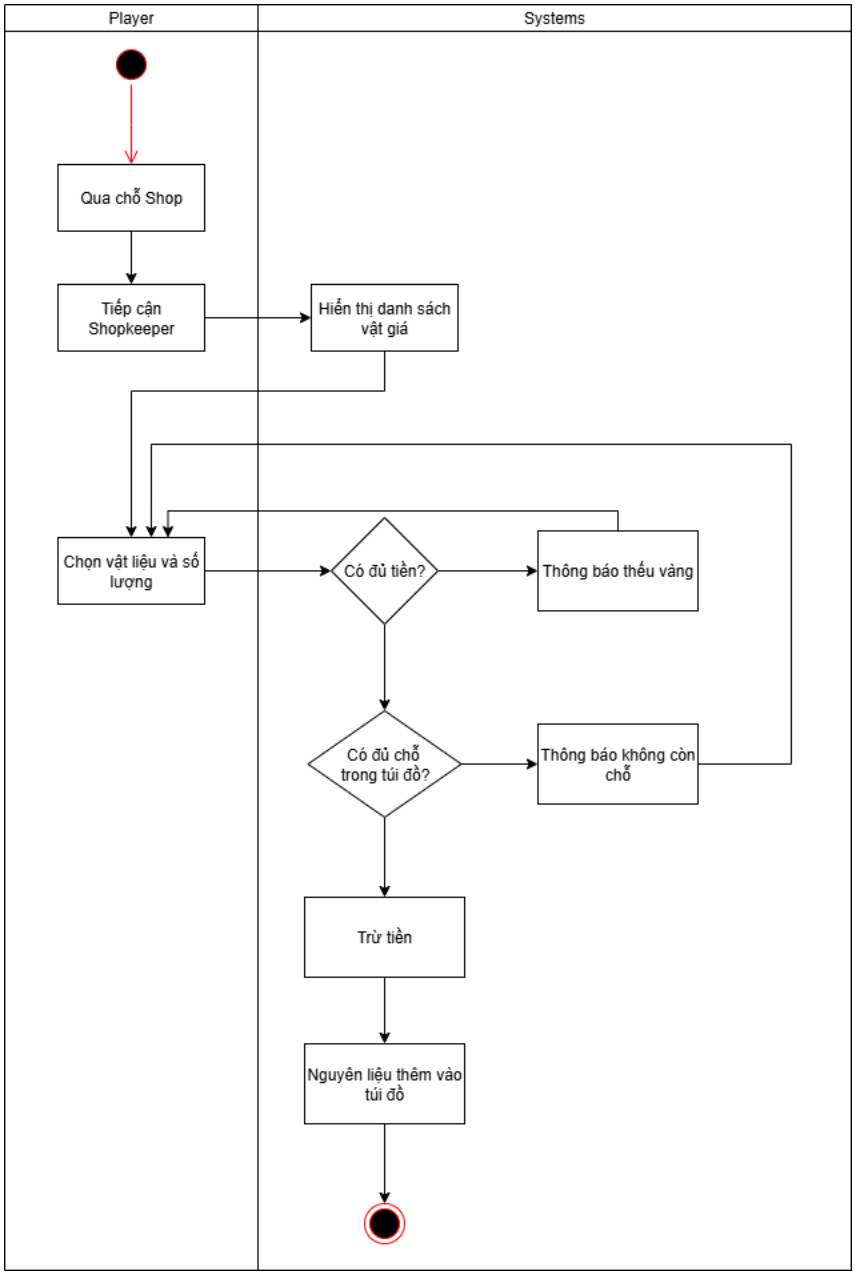
2.1 Activity Diagrams

New Game Activity



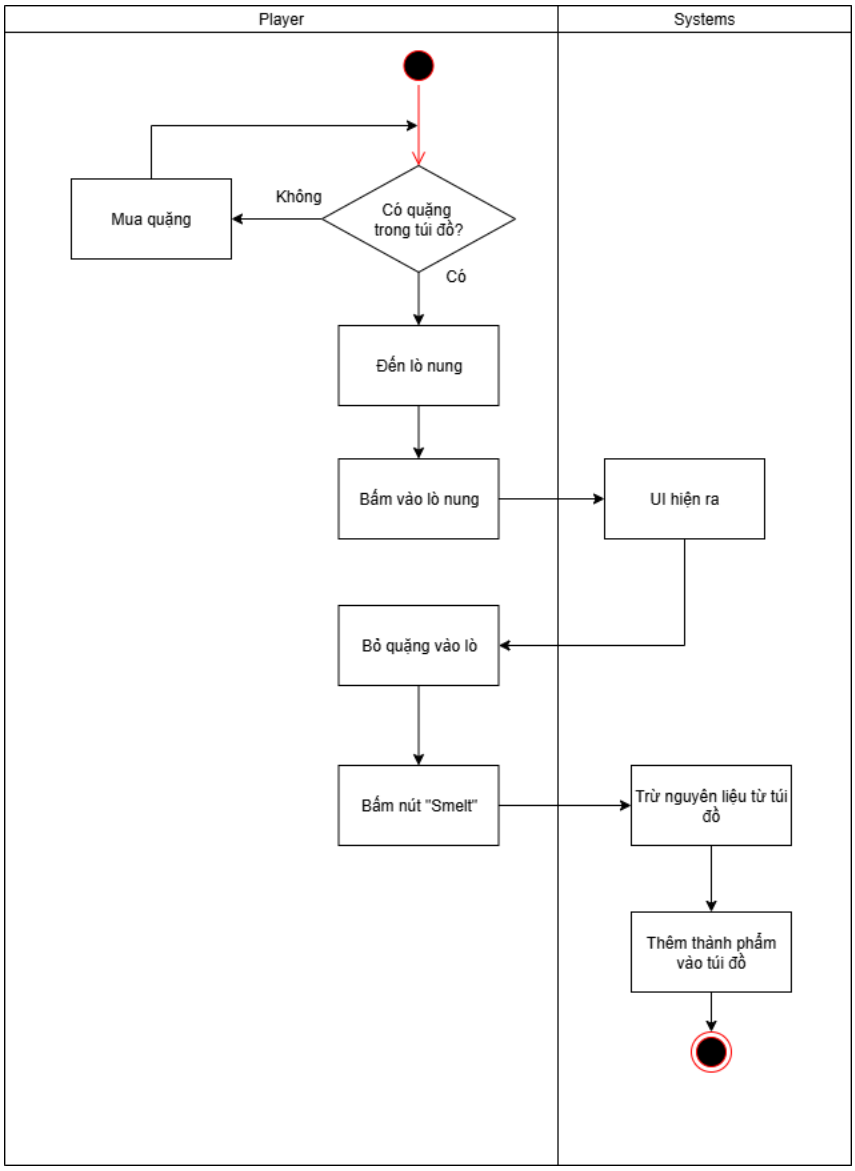
Mô tả: Người chơi chọn New Game, hệ thống kiểm tra có file save cũ không. Nếu có, hiển thị cảnh báo dữ liệu cũ sẽ bị ghi đè và chờ người chơi xác nhận. Sau khi xác nhận hoặc không có save cũ, hệ thống khởi tạo dữ liệu game mới (Inventory rỗng, vàng = 100, Quest Log rỗng). Cuối cùng, hệ thống kiểm tra lưu dữ liệu thành công, nếu có lỗi thì hiển thị thông báo lỗi và quay lại Main Menu, nếu thành công thì vào game.

Buy Material Activity



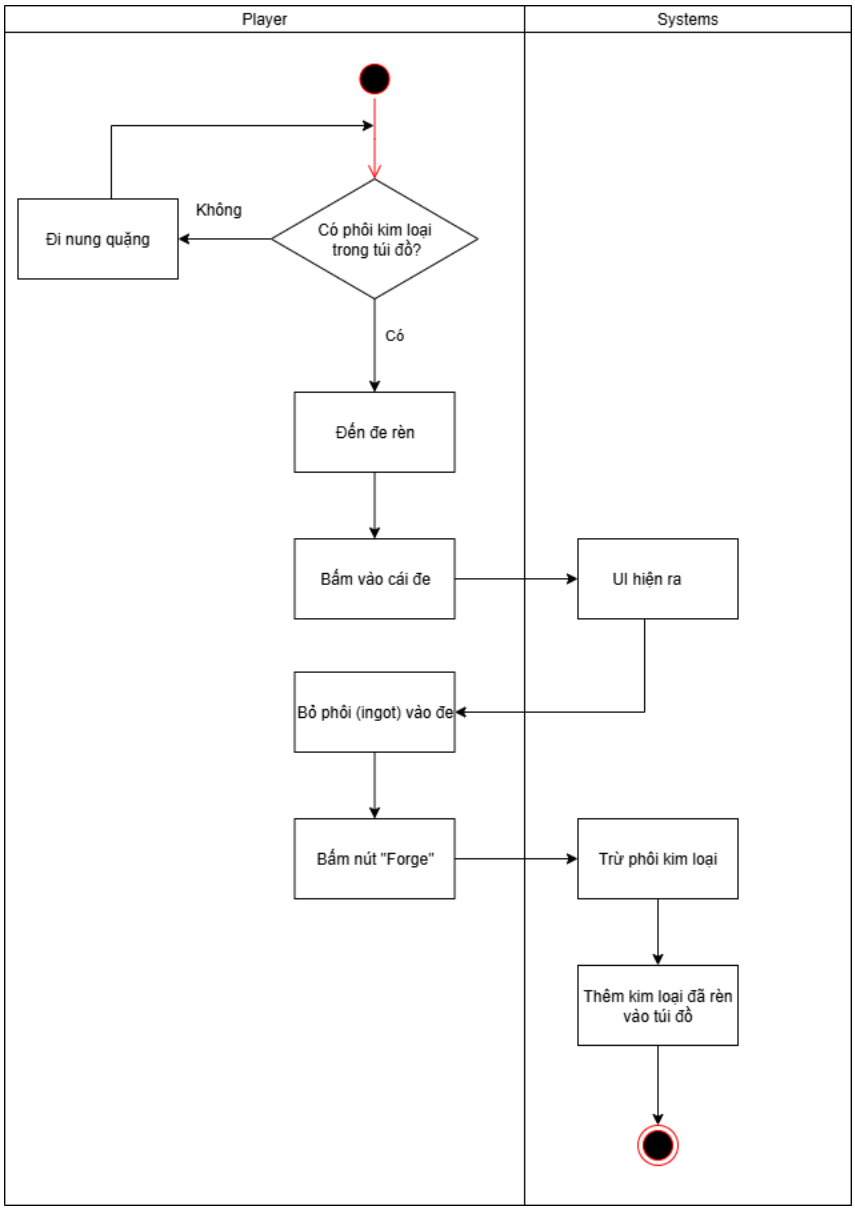
Mô tả: Người chơi qua chỗ Shop, tiếp cận Shopkeeper để hệ thống hiển thị danh sách vật giá. Người chơi chọn vật liệu và số lượng, hệ thống kiểm tra đủ tiền và đủ chỗ trong túi đồ. Sau khi kiểm tra thành công, hệ thống trừ tiền và thêm nguyên liệu vào túi đồ.

Smelting Activity



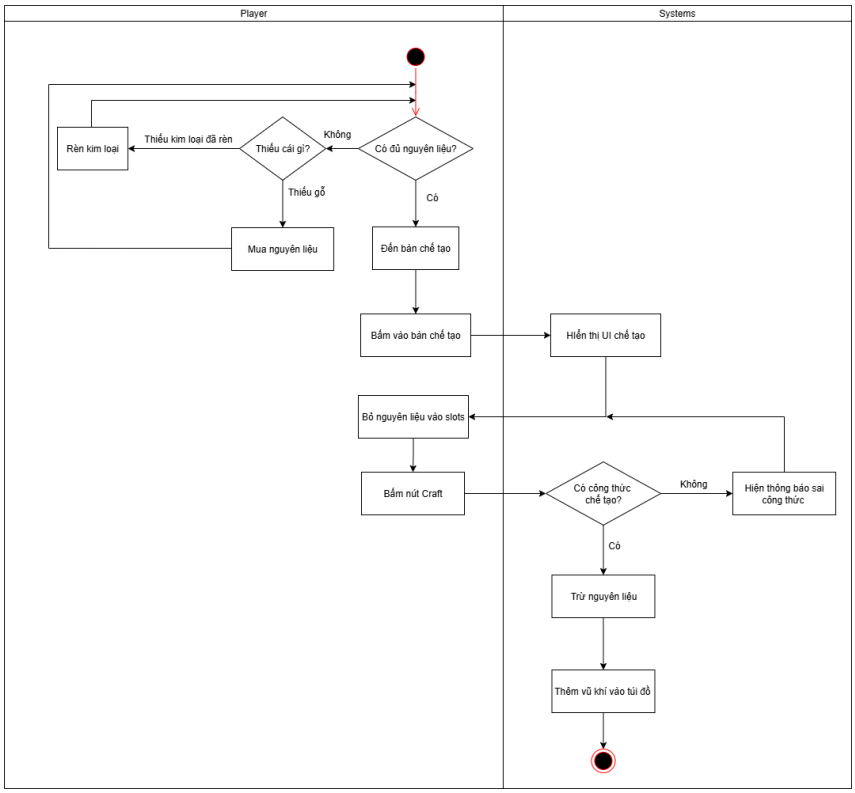
Mô tả: Người chơi kiểm tra có phôi kim loại trong túi đồ không. Nếu không thì đi nùng quặng. Nếu có, đến đế rèn, bấm vào cái đế để UI hiện ra, bỏ phôi (ingot) vào đế, bấm nút "Forge" để hệ thống trừ phôi kim loại và thêm kim loại đã rèn vào túi đồ.

Forging Activity



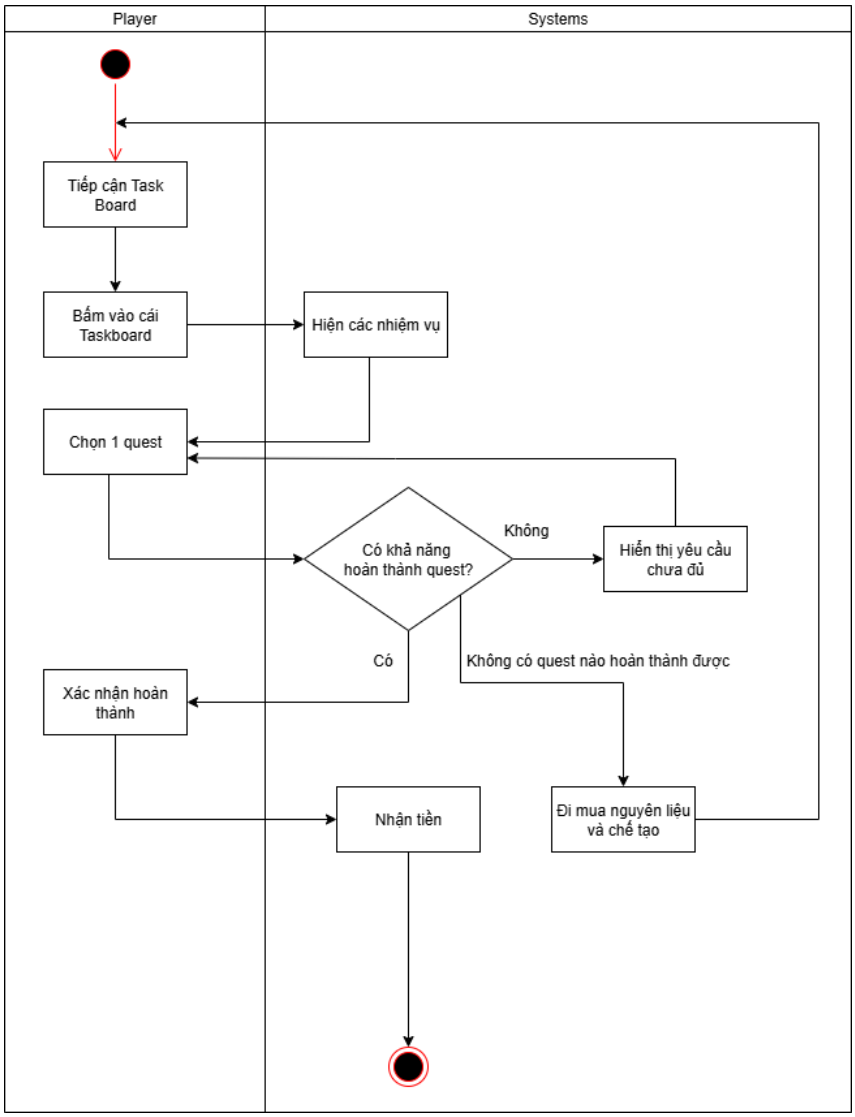
Mô tả: Quy trình rèn vũ khí bắt đầu khi người chơi kiểm tra quặng trong túi đồ. Nếu không đủ quặng, người chơi cần mua thêm. Sau đó, người chơi đến lò rèn, bấm vào lò để UI hiện ra, bỏ quặng vào lò, bấm nút "Smelt" để hệ thống trừ nguyên liệu từ túi đồ và thêm thành phẩm vào túi đồ.

Crafting Activity



Mô tả: Quy trình bắt đầu khi hệ thống kiểm tra đủ nguyên liệu. Người chơi có thể chọn thiếu kim loại để rèn kim loại hoặc thiếu gỗ để mua nguyên liệu. Sau đó đến bàn chế tạo, bấm vào để UI hiện ra, bỏ nguyên liệu vào slots, bấm nút Craft. Hệ thống kiểm tra có công thức chế tạo không, nếu có thì trừ nguyên liệu và thêm vũ khí vào túi đồ.

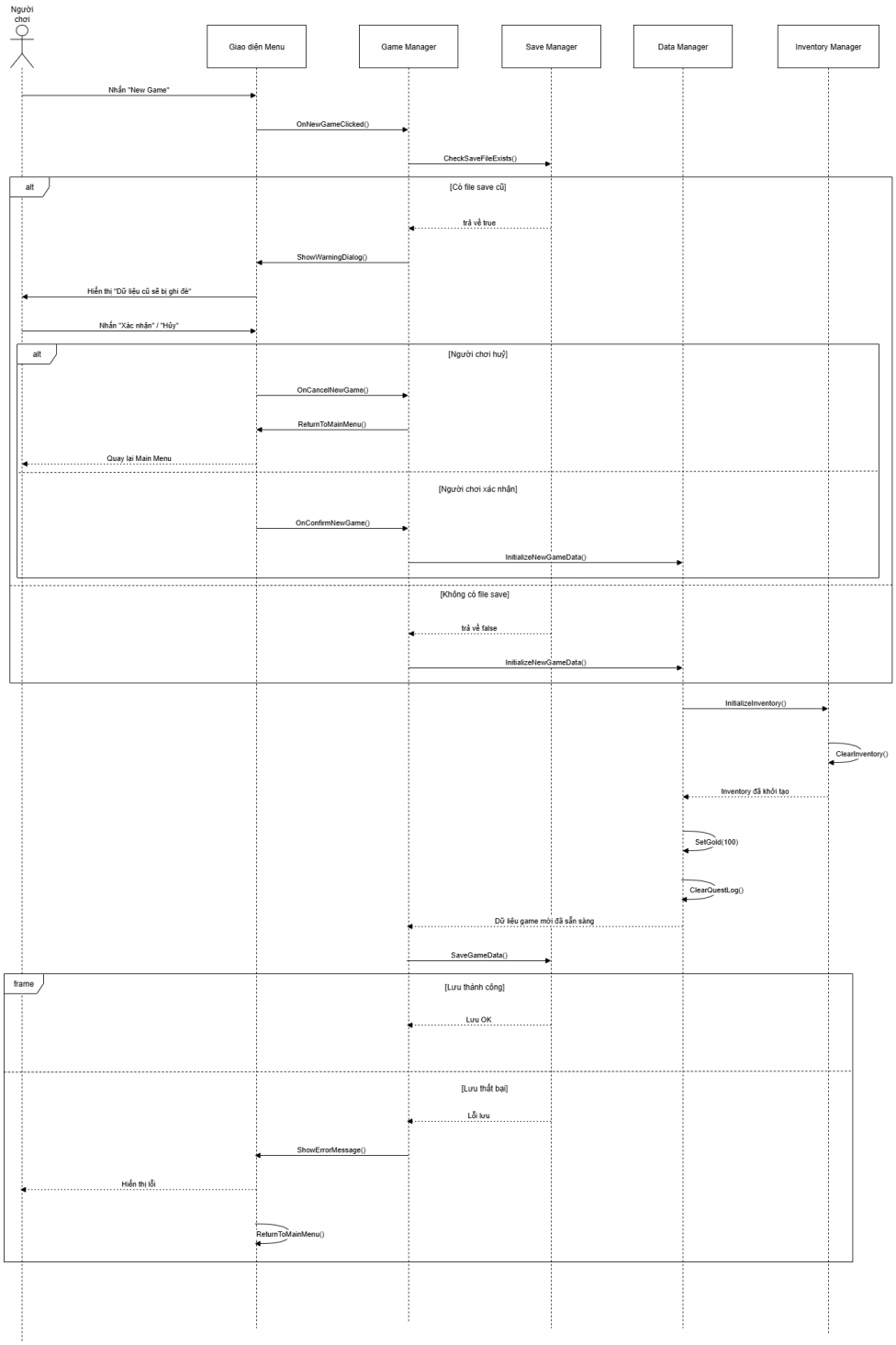
Complete Quest



Mô tả: Người chơi tiếp cận Task Board, bấm vào để hệ thống hiển thị các nhiệm vụ khả dụng. Người chơi chọn một quest, hệ thống kiểm tra khả năng hoàn thành. Nếu không đủ điều kiện, hiển thị yêu cầu chưa đủ. Nếu đủ điều kiện, người chơi xác nhận hoàn thành và hệ thống nhận tiền, đi mua nguyên liệu và chế tạo vật phẩm.

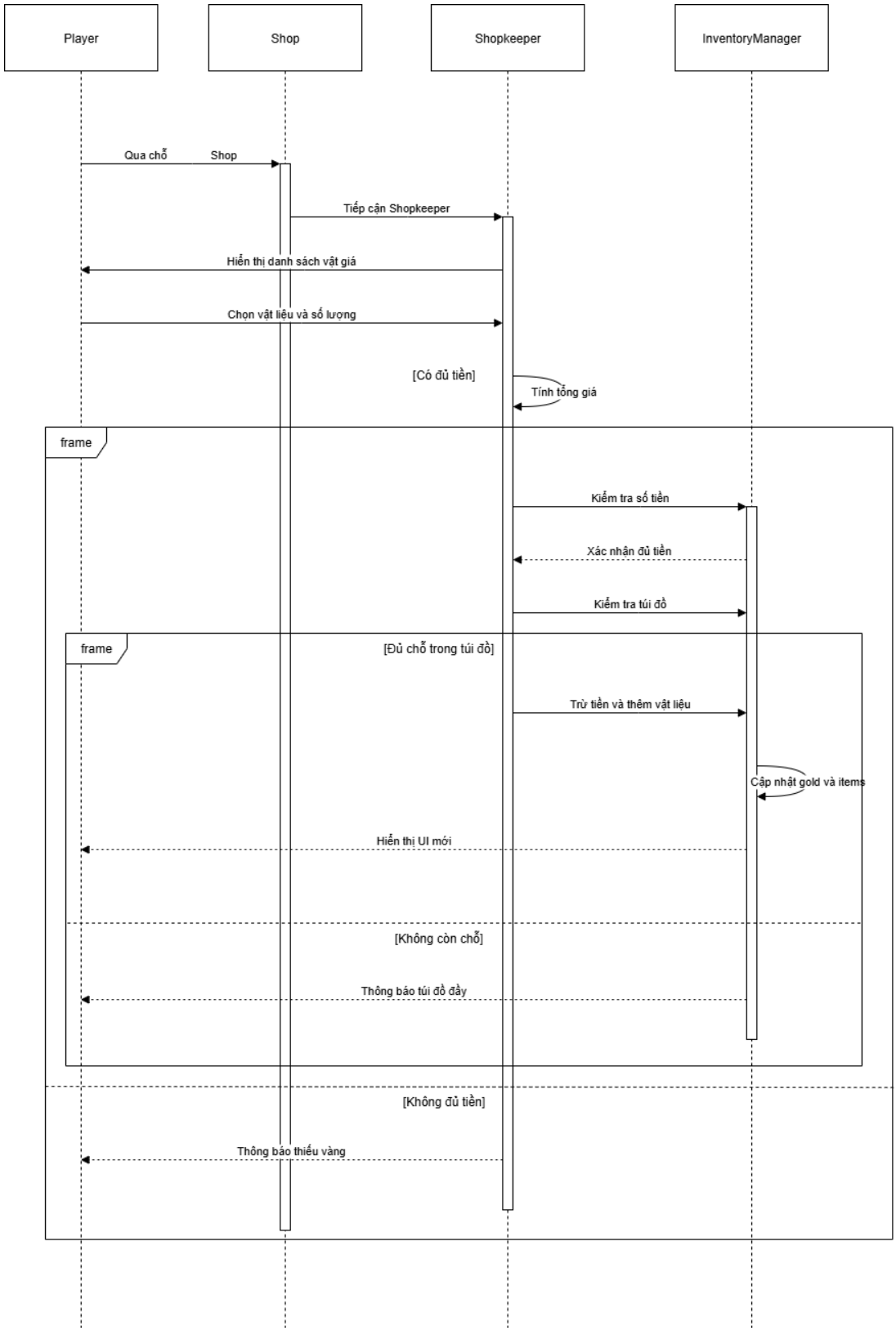
2.2 Sequence Diagrams

New Game Sequence



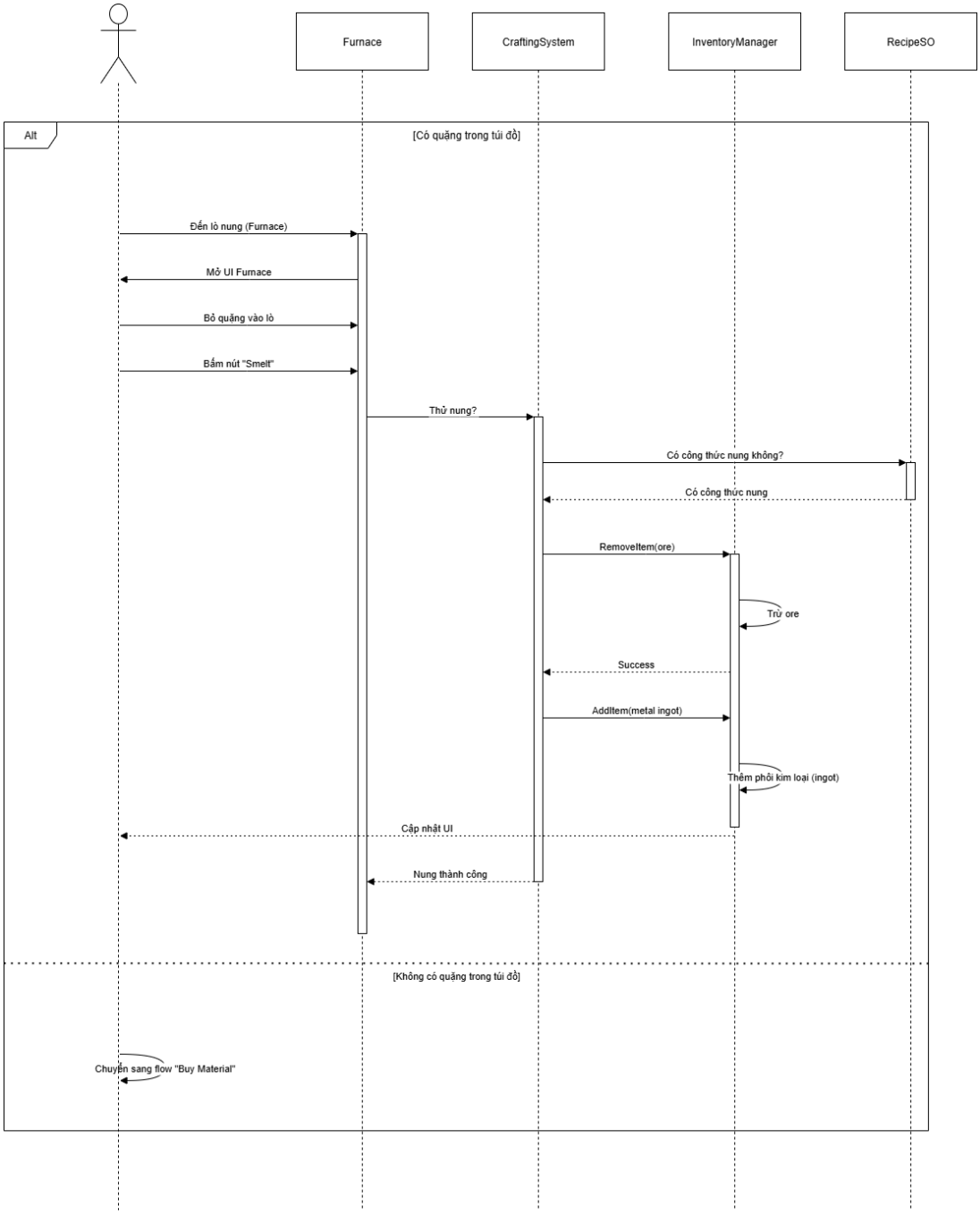
Mô tả: Người chơi nhấn "New Game" từ giao diện Menu. Game Manager gọi OnNewGameClicked() và kiểm tra save file cũ với Save Manager. Trong điều kiện "[Có file save cũ]", hiển thị cảnh báo "Dữ liệu cũ sẽ bị ghi đè" và chờ người chơi xác nhận "Xác nhận" hoặc "Hủy". Nếu người chơi xác nhận hoặc trong điều kiện "[Người chơi không có file save]", Game Manager gọi OnCancelNewGame() và trả về Main Menu. Trong điều kiện "[Người chơi xác nhận]", Game Manager gọi OnConfirmNewGame() và khởi tạo dữ liệu game mới với Data Manager. Data Manager gọi InitializeNewGameData() để khởi tạo Inventory Manager (ClearInventory(), SetGold(100), ClearQuestLog()). Trong frame "[Lưu thành công]", Game Manager lưu dữ liệu game mới (SaveGameData()) với Save Manager, nhận OK. Trong frame "[Lưu thất bại]", hiển thị thông báo lỗi (ShowErrorMessage()) và quay lại Main Menu (ReturnToMainMenu()).

Buy Material Sequence



Mô tả: Người chơi qua chỗ Shop và tiếp cận Shopkeeper. Shop hiển thị danh sách vật giá cho người chơi. Người chơi chọn vật liệu và số lượng. Trong frame "[Có đủ tiền]", Shopkeeper tính tổng giá, sau đó kiểm tra số tiền với InventoryManager. Nếu đủ tiền (frame "[Đủ chỗ trong túi đồ]"), InventoryManager trừ tiền và thêm vật liệu, cập nhật gold và items, sau đó hiển thị UI mới cho người chơi. Nếu không đủ chỗ, thông báo túi đồ đầy. Nếu không đủ tiền, thông báo thiếu vàng.

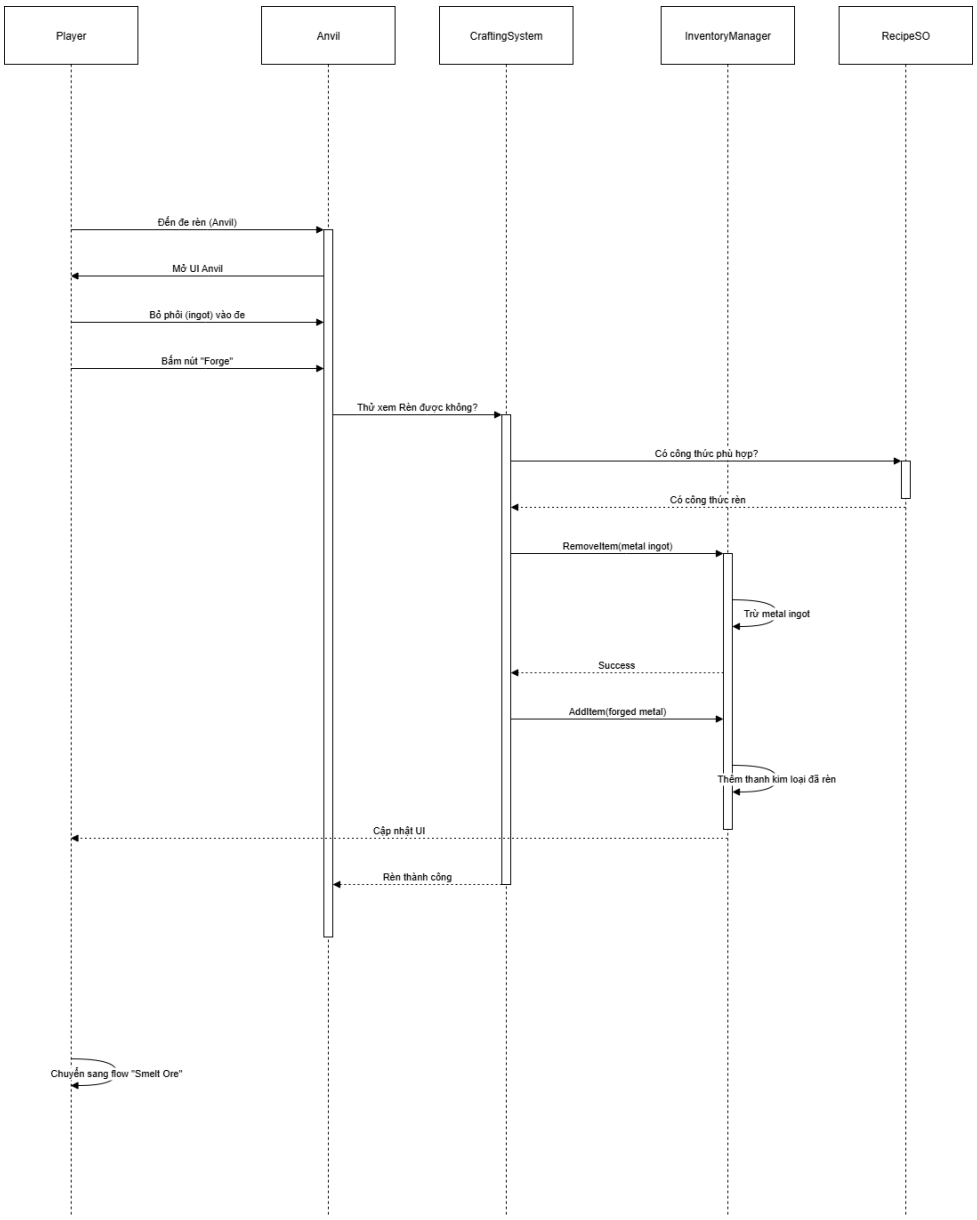
Furnace Sequence



Mô tả: Trong điều kiện "[Có quặng trong túi đồ]", người chơi đến lò nùng (Furnace), mở UI Furnace, bỏ quặng vào lò và bấm nút "Smelt". Furnace gửi yêu cầu thử nùng đến CraftingSystem, CraftingSystem kiểm tra có công thức nùng không với RecipeSO. Nếu có công thức nùng, CraftingSystem yêu cầu InventoryManager xóa quặng (RemoveItem), trừ ore, trả về Success, sau đó thêm kim loại đã nùng (metal ingot) vào túi đồ. Cuối cùng

cập nhật UI và thông báo nùng thành công. Nếu không có quặng trong túi đồ, chuyển sang flow "Buy Material".

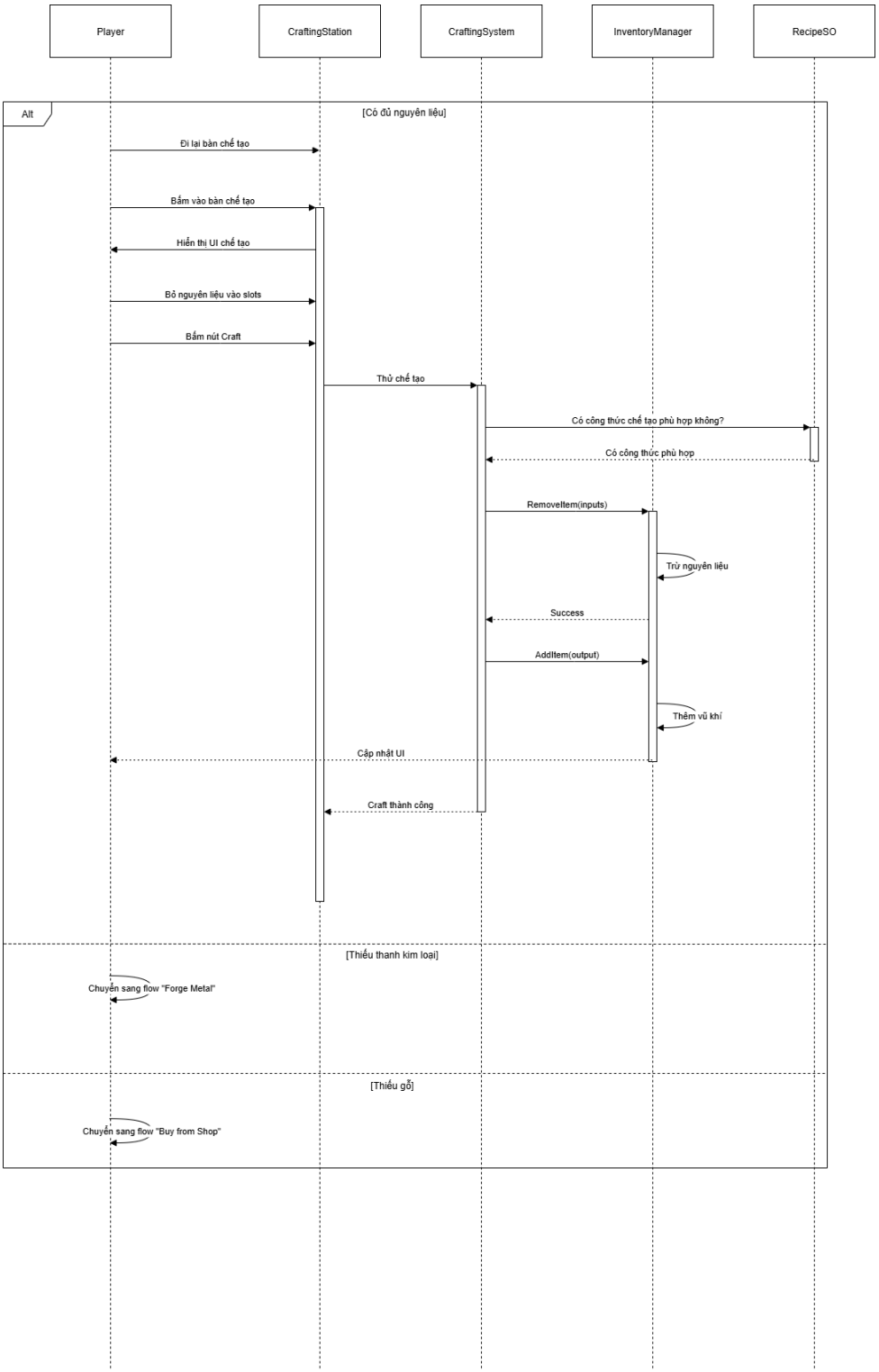
Anvil Sequence



Mô tả: Người chơi đến đế rèn (Anvil), mở UI Anvil, bỏ phôi (ingot) vào đế và bấm nút "Forge". Anvil gửi yêu cầu thử xem rèn được không đến CraftingSystem. CraftingSystem kiểm tra có công thức phù hợp với RecipeSO. Nếu có công thức phù hợp, CraftingSystem yêu cầu InventoryManager xóa metal ingot (RemoveItem), trừ metal ingot, trả về Success,

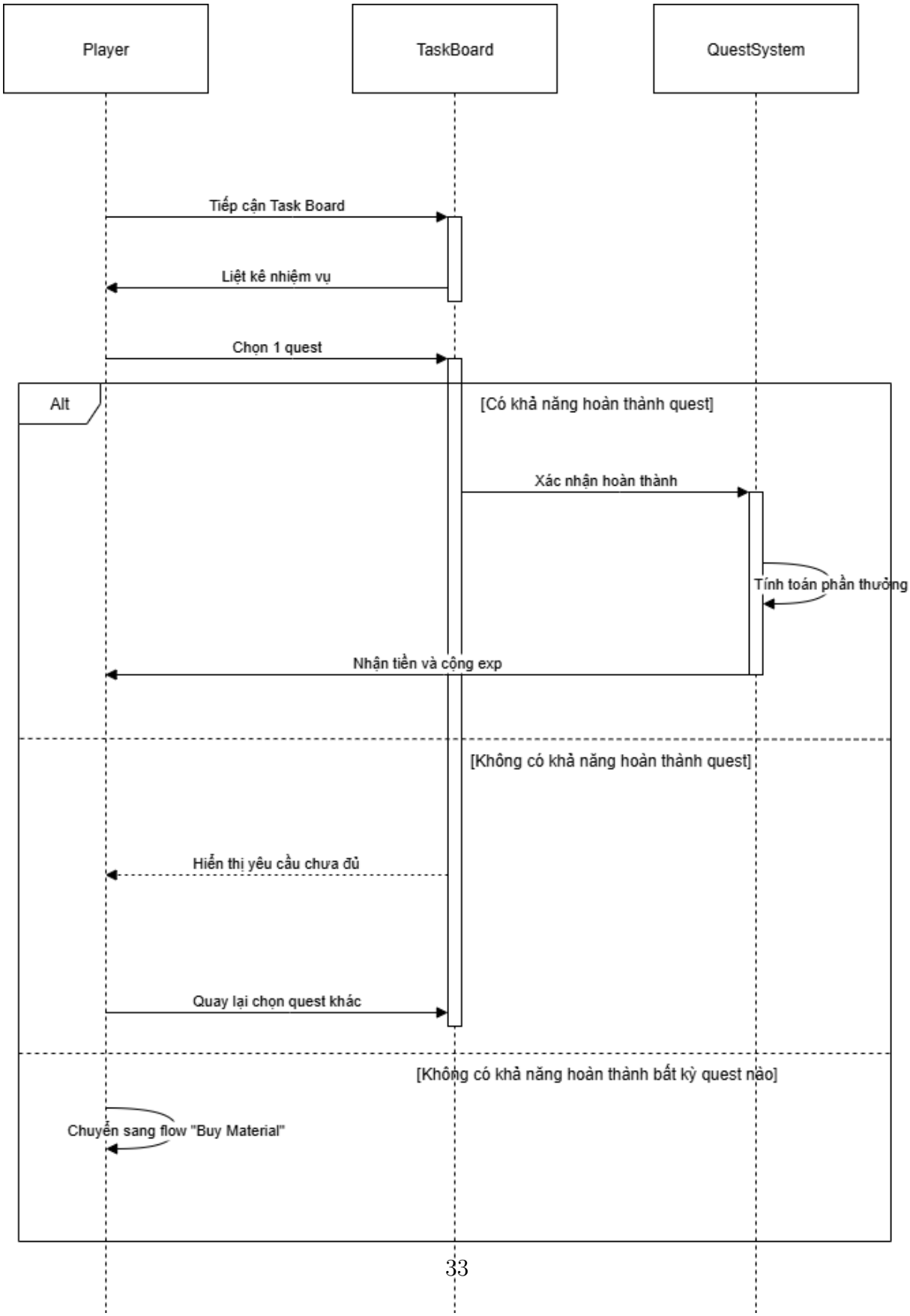
sau đó thêm kim loại đã rèn (forged metal) vào túi đồ. Cuối cùng cập nhật UI và thông báo rèn thành công. Sau khi hoàn thành, chuyển sang flow "Smelt Ore".

Crafting Sequence



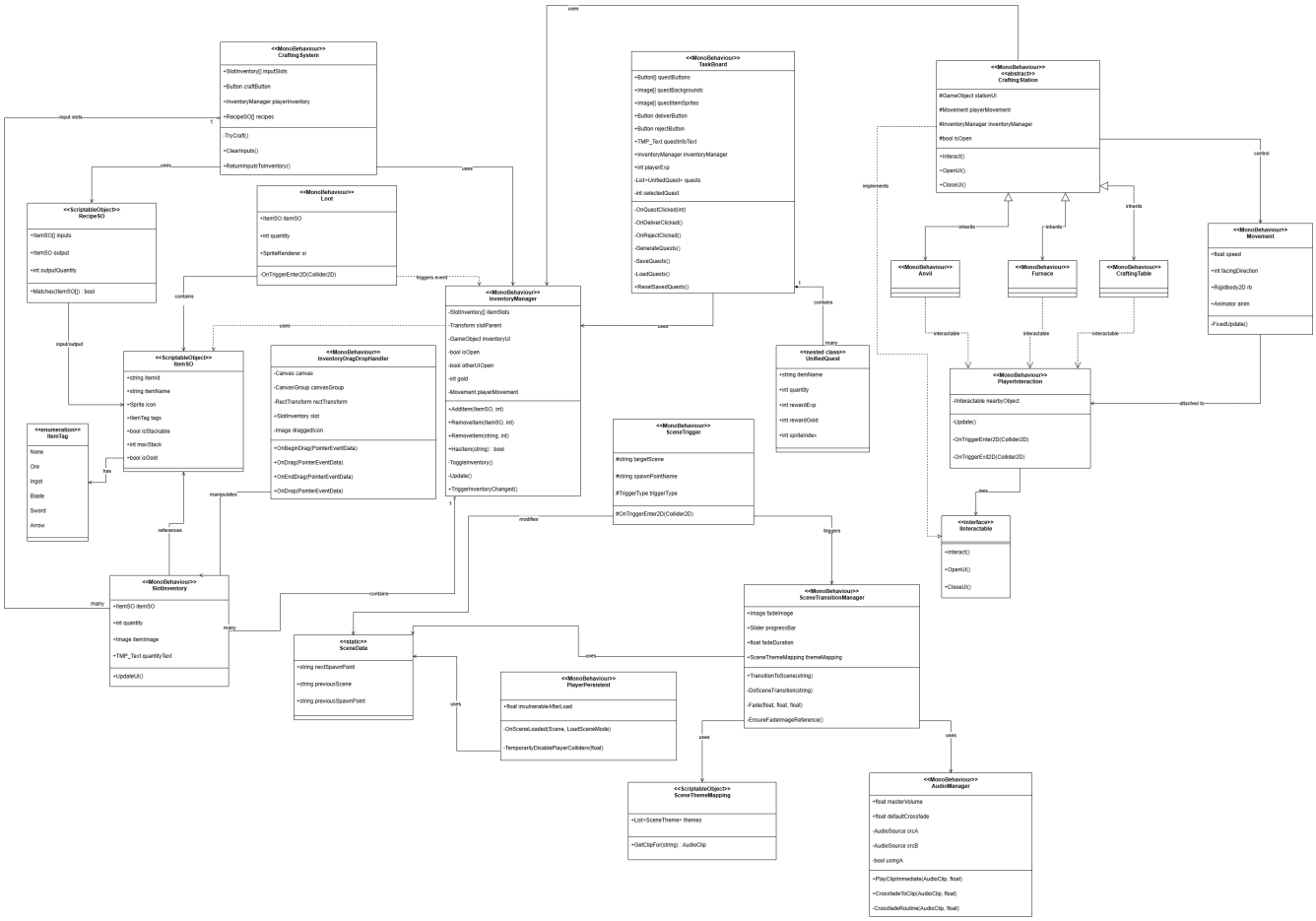
Mô tả: Trong điều kiện "[Có đủ nguyên liệu]", người chơi đi tại bàn chế tạo, bấm vào bàn chế tạo để hiển thị UI chế tạo, bỏ nguyên liệu vào slots và bấm nút Craft. CraftingStation gửi yêu cầu thử chế tạo đến CraftingSystem. CraftingSystem kiểm tra có công thức chế tạo phù hợp không với RecipeSO. Nếu có công thức phù hợp, CraftingSystem yêu cầu InventoryManager xóa nguyên liệu (RemoveItem), trừ nguyên liệu, trả về Success, sau đó thêm vũ khí (AddItem) vào túi đồ. Cuối cùng cập nhật UI và thông báo Craft thành công. Nếu thiếu thành kim loại, chuyển sang flow "Forge Metal". Nếu thiếu gỗ, chuyển sang flow "Buy from Shop".

Complete Quest Sequence



Mô tả: Người chơi tiếp cận Task Board, TaskBoard liệt kê nhiệm vụ cho người chọi. Người chơi chọn 1 quest. Trong điều kiện Alt "[Có khả năng hoàn thành quest]", TaskBoard xác nhận hoàn thành với QuestSystem, QuestSystem tính toán phần thưởng và trả về nhận tiền và cộng exp cho người chơi. Trong điều kiện "[Không có khả năng hoàn thành quest]", TaskBoard hiển thị yêu cầu chưa đủ cho người chơi và người chơi quay lại chọn quest khác. Trong điều kiện "[Không có khả năng hoàn thành bất kỳ quest nào]", chuyển sang flow "Buy Material".

2.3 Class Diagram



2.3.1 Mô tả các phương thức trong Class Diagram

CraftingSystem

Hệ thống quản lý chế tạo vật phẩm.

- **TryCraft()**: Thử thực hiện chế tạo bằng cách kiểm tra công thức và nguyên liệu trong input slots
- **ClearInputs()**: Xóa tất cả nguyên liệu trong các ô input
- **ReturnInputsToInventory()**: Trả lại tất cả nguyên liệu từ input slots về túi đồ của người chơi

Loot

Quản lý vật phẩm rơi ra trên bản đồ.

- **OnTriggerEnter2D(Collider2D)**: Phát hiện va chạm với người chơi và tự động thêm vật phẩm vào túi đồ

TaskBoard

Quản lý bảng nhiệm vụ và giao dịch với người chơi.

- **OnQuestClicked(int)**: Xử lý khi người chơi click chọn một nhiệm vụ từ danh sách
- **OnDeliverClicked()**: Xử lý khi người chơi nhấn nút giao vật phẩm để hoàn thành nhiệm vụ
- **OnRejectClicked()**: Xử lý khi người chơi từ chối/hủy nhiệm vụ đã chọn
- **GenerateQuests()**: Tạo ra danh sách nhiệm vụ ngẫu nhiên hoặc theo cấu hình
- **SaveQuests()**: Lưu trạng thái nhiệm vụ hiện tại vào file
- **LoadQuests()**: Tải trạng thái nhiệm vụ từ file đã lưu
- **ResetSavedQuests()**: Xóa tất cả dữ liệu nhiệm vụ đã lưu và tạo mới

InventoryManager

Quản lý túi đồ của người chơi.

- **AddItem(ItemSO, int)**: Thêm vật phẩm vào túi đồ với số lượng chỉ định
- **RemoveItem(ItemSO, int)**: Xóa vật phẩm khỏi túi đồ theo đối tượng ItemSO
- **RemoveItem(string, int)**: Xóa vật phẩm khỏi túi đồ theo itemId

- **HasItem(string) : bool**: Kiểm tra xem túi đồ có chứa vật phẩm với itemId cụ thể không
- **ToggleInventory()**: Bật/tắt hiển thị UI túi đồ
- **Update()**: Cập nhật logic hàng frame (kiểm tra input mở túi đồ)
- **TriggerInventoryChanged()**: Kích hoạt sự kiện thông báo túi đồ đã thay đổi để cập nhật UI

InventoryDragDropHandler

Xử lý kéo thả vật phẩm trong túi đồ.

- **OnBeginDrag(PointerEventData)**: Bắt đầu kéo vật phẩm, tạo icon ảo theo chuột
- **OnDrag(PointerEventData)**: Cập nhật vị trí icon ảo theo chuột trong khi kéo
- **OnEndDrag(PointerEventData)**: Kết thúc kéo, xử lý logic thả vật phẩm
- **OnDrop(PointerEventData)**: Nhận vật phẩm được thả vào ô này và hoán đổi/sắp xếp

SlotInventory

Đại diện cho một ô trong túi đồ.

- **UpdateUI()**: Cập nhật hiển thị icon, số lượng của vật phẩm trong ô

ItemSO (ScriptableObject)

Dữ liệu tĩnh của vật phẩm, không có method nào.

RecipeSO (ScriptableObject)

Công thức chế tạo.

- **Matches(ItemSO[]) : bool**: Kiểm tra xem mảng vật phẩm input có khớp với công thức này không

SceneTrigger

Trigger chuyển cảnh khi người chơi tiếp cận.

- **OnTriggerEnter2D(Collider2D)**: Phát hiện người chơi và kích hoạt chuyển cảnh

PlayerPersistent

Quản lý persistence của người chơi qua các scene.

- **OnSceneLoaded(Scene, LoadSceneMode)**: Được gọi khi scene mới được tải, xử lý spawn position
- **TemporarilyDisablePlayerColliders(float)**: Tạm thời vô hiệu hóa collider để tránh va chạm khi vừa spawn

SceneTransitionManager

Quản lý chuyển cảnh với hiệu ứng fade.

- **TransitionToScene(string)**: Bắt đầu quá trình chuyển sang scene mới
- **DoSceneTransition(string)**: Coroutine thực hiện fade out, load scene, fade in
- **Fade(float, float, float)**: Coroutine điều khiển độ trong suốt của fade image
- **EnsureFadeImageReference()**: Đảm bảo fade image được tham chiếu đúng

SceneThemeMapping (ScriptableObject)

Ánh xạ scene với nhạc nền.

- **GetClipFor(string) : AudioClip**: Lấy AudioClip tương ứng với tên scene

AudioManager

Quản lý phát nhạc nền.

- **PlayClipImmediate(AudioClip, float)**: Phát nhạc ngay lập tức với volume chỉ định
- **CrossfadeToClip(AudioClip, float)**: Chuyển đổi mượt mà (crossfade) sang bản nhạc mới
- **CrossfadeRoutine(AudioClip, float)**: Coroutine thực hiện crossfade giữa 2 audio source

Movement

Điều khiển di chuyển của nhân vật.

- **FixedUpdate()**: Cập nhật vật lý di chuyển mỗi physics frame

CraftingStation (Abstract)

Class trừu tượng cho các trạm chế tạo.

- **Interact()**: Được gọi khi người chơi tương tác với trạm
- **OpenUI()**: Mở giao diện UI của trạm chế tạo
- **CloseUI()**: Đóng giao diện UI của trạm chế tạo

CraftingTable, Furnace, Anvil

Các trạm chế tạo cụ thể, kế thừa từ CraftingStation. Không có method riêng, sử dụng method từ class cha.

PlayerInteraction

Xử lý tương tác của người chơi với các đối tượng.

- **Update()**: Kiểm tra input tương tác mỗi frame
- **OnTriggerEnter2D(Collider2D)**: Phát hiện đối tượng có thể tương tác khi người chơi đến gần
- **OnTriggerExit2D(Collider2D)**: Xóa tham chiếu đối tượng khi người chơi đi xa

IInteractable (Interface)

Interface cho các đối tượng có thể tương tác.

- **Interact()**: Thực hiện hành động tương tác
- **OpenUI()**: Mở giao diện người dùng
- **CloseUI()**: Đóng giao diện người dùng

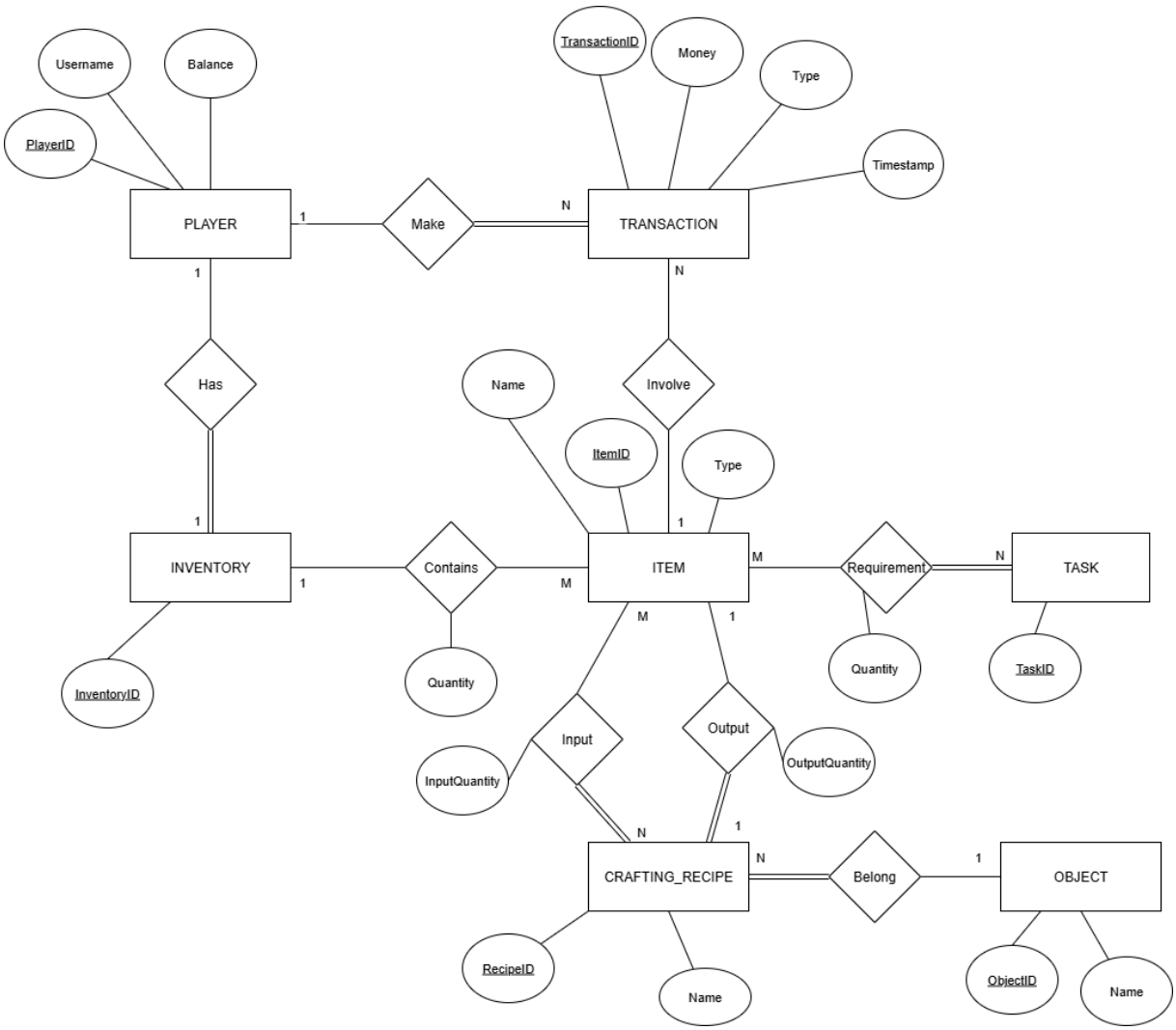
SceneData (Static)

Lưu trữ dữ liệu tĩnh về chuyển cảnh (spawn point, scene trước đó). Không có method, chỉ có thuộc tính tĩnh.

3 Thiết kế kiến trúc

3.1 ERD và Relational Schema

3.1.1 ER Diagram



3.1.2 Relational Schema

Thực thể chính

PLAYER

```sql
PLAYER (
    PlayerID INT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL,
    Balance DECIMAL(10,2) DEFAULT 0
);
```

Người chơi, có tài khoản, có tiền.

ITEM

```sql
ITEM (
    ItemID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Type VARCHAR(30)
);
```

Vật phẩm trong game, có tên và loại.

OBJECT

```sql
OBJECT (
    ObjectID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL
);
```

Vật thể tương tác trong thế giới game.

Túi đồ (Inventory)

INVENTORY

```sql
INVENTORY (
    InventoryID INT PRIMARY KEY,
    PlayerID INT NOT NULL,
    FOREIGN KEY (PlayerID) REFERENCES PLAYER(PlayerID)
);
```

Một người chơi có thể có nhiều kho chứa.

CONTAINS

```sql
CONTAINS (
    InventoryID INT NOT NULL,
```

```
    ItemID INT NOT NULL,
    Quantity INT DEFAULT 1,
    PRIMARY KEY (InventoryID, ItemID),
    FOREIGN KEY (InventoryID) REFERENCES INVENTORY(InventoryID),
    FOREIGN KEY (ItemID) REFERENCES ITEM(ItemID)
);
```

Mỗi dòng mô tả một vật phẩm và số lượng trong kho.

Quan hệ Contains: INVENTORY 1 — M ITEM

Hệ thống Crafting

CRAFTING_RECIPE

```
CRAFTING_RECIPE (
    RecipeID INT PRIMARY KEY,
    Name VARCHAR(50),
    OutputItemID INT NOT NULL,
    ObjectID INT NOT NULL,
    FOREIGN KEY (OutputItemID) REFERENCES ITEM(ItemID),
    FOREIGN KEY (ObjectID) REFERENCES OBJECT(ObjectID)
);
```

Mỗi công thức tạo ra một vật phẩm tại một object.

INPUT

```
INPUT (
    RecipeID INT NOT NULL,
    ItemID INT NOT NULL,
    InputQuantity INT NOT NULL,
    PRIMARY KEY (RecipeID, ItemID),
    FOREIGN KEY (RecipeID) REFERENCES CRAFTING_RECIPE(RecipeID),
    FOREIGN KEY (ItemID) REFERENCES ITEM(ItemID)
);
```

Danh sách nguyên liệu đầu vào.

Quan hệ Input: CRAFTING_RECIPE N — M ITEM

Nhiệm vụ (Task System)

TASK

```
TASK (
    TaskID INT PRIMARY KEY,
```

```
    PlayerID INT NOT NULL,
    Name VARCHAR(100),
    Status VARCHAR(20) DEFAULT 'Pending',
    Reward DECIMAL(10,2),
    FOREIGN KEY (PlayerID) REFERENCES PLAYER(PlayerID)
);
```

Nhiệm vụ người chơi có thể nhận.

REQUIREMENT

```
REQUIREMENT (
    TaskRequirementID INT PRIMARY KEY,
    TaskID INT NOT NULL,
    ItemID INT NOT NULL,
    Quantity INT NOT NULL,
    FOREIGN KEY (TaskID) REFERENCES TASK(TaskID),
    FOREIGN KEY (ItemID) REFERENCES ITEM(ItemID)
);
```

Vật phẩm cần nộp để hoàn thành nhiệm vụ.

Giao dịch (Transaction System)

TRANSACTION

```
TRANSACTION (
    TransactionID INT PRIMARY KEY,
    PlayerID INT NOT NULL,
    ItemID INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Type VARCHAR(10) CHECK (Type IN ('BUY','SELL')),
    DateTime DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (PlayerID) REFERENCES PLAYER(PlayerID),
    FOREIGN KEY (ItemID) REFERENCES ITEM(ItemID)
);
```

Lưu lại hoạt động mua bán vật phẩm của người chơi.

Tổng kết các quan hệ chính

Relationship	Cardinality	Total/Partial
PLAYER – INVENTORY	1–N	Partial / Total
INVENTORY – ITEM	M–N	Partial / Partial
OBJECT – CRAFTING_RECIPE	1–N	Partial / Total
CRAFTING_RECIPE – RECIPE (INPUT)	1–N	Partial / Total
CRAFTING_RECIPE – RECIPE (OUTPUT)	1–1	Partial / Total
RECIPE_INPUT – ITEM	N–1	Partial / Total
PLAYER – TASK	1–N	Partial / Total
TASK – ITEM (REQUIREMENT)	1–N	Partial / Total
PLAYER – TRANSACTION	1–N	Partial / Total
TRANSACTION – ITEM	N–1	Partial / Total

Table 3.1: Các mối quan hệ giữa các thực thể

3.2 Phương pháp tiếp cận kiến trúc để triển khai hệ thống

3.2.1 Tổng quan kiến trúc

Hệ thống được triển khai theo kiến trúc **Client-based Game Architecture** trong Unity, nơi toàn bộ logic xử lý dữ liệu (transactions, crafting, inventory, tasks, v.v.) nằm phía client.

Về cấu trúc phần mềm, ứng dụng sẽ tuân theo mô hình **Entity-Component-System (ECS)/Object-Oriented** tùy phần Unity dùng, nhưng chủ yếu tổ chức theo:

- **Data Layer (Model):** ánh xạ từ ERD sang các lớp C# (ScriptableObject hoặc class).
- **Logic Layer (Controllers/Managers):** điều khiển gameplay (InventoryManager, CraftingManager, v.v.).
- **Presentation Layer (UI):** hiển thị trạng thái lên màn hình.

Cách phân lớp này giúp dễ bảo trì, mở rộng, và tái sử dụng.

3.2.2 Data Layer — Mapping ERD → Unity C#

Dựa trên ERD, các thực thể được chuyển thành các lớp dữ liệu trong Unity:

PLAYER

- Lưu bằng class Player hoặc ScriptableObject.
- Bao gồm ID, Username, Balance.
- Có tham chiếu đến Inventory.

INVENTORY

- Chứa danh sách ItemStack (Item + quantity).
- Được quản lý bởi InventoryManager.

ITEM

- Được thiết kế bằng **ScriptableObject** (chuẩn nhất trong Unity).
- Lưu các thông tin tĩnh: ItemID, Name, Type.

TRANSACTION

- Lưu dưới dạng log hoặc list trong Player.
- Ghi lại thay đổi tiền hoặc vật phẩm.

CRAFTING_RECIPE

- Cũng là ScriptableObject.
- Gồm:
  - danh sách InputItems (Item + qty)
  - OutputItem (Item + qty)
  - tên recipe
  - Object/Workbench mà recipe thuộc về

TASK

- Class/ScriptableObject mô tả nhiệm vụ, bao gồm yêu cầu Item.

OBJECT

- Các đối tượng tương tác trong thế giới (Crafting Table, Furnace, v.v.)
- Mỗi object tham chiếu tới các recipe mà nó hỗ trợ.

3.2.3 Mapping Relationship

- Quan hệ nhiều–nhiều chuyển thành:
  - List<ItemRequirement> trong Recipe
  - List<ItemRequirement> trong Task
    (ItemRequirement = Item + quantity)
- Quan hệ 1–1 hoặc 1–N chuyển thành:
  - Thuộc tính tham chiếu trong class (Player.Inventory, Recipe.OutputItem, v.v.)

3.2.4 Logic Layer — Gameplay Managers

Tầng này gồm các script xử lý logic chính của game.

InventoryManager

- Thêm/xóa item
- Kiểm tra số lượng
- Tương tác với crafting và task

CraftingManager

- Nhận yêu cầu crafting
- Kiểm tra xem inventory có đủ nguyên liệu không
- Nếu đủ: trừ input, thêm output
- Ghi log Transaction nếu cần

TransactionManager

- Ghi và quản lý tất cả giao dịch của Player
- GUI hiển thị lịch sử khi cần

TaskManager

- Theo dõi tiến độ nhiệm vụ
- Kiểm tra khi Player nhận đủ Item yêu cầu

ObjectInteraction

- Khi Player tương tác vào một Object
- Mở UI recipe tương ứng object đó

3.2.5 Presentation Layer — UI

- UI hiển thị thông tin player, tiền, vật phẩm
- UI crafting theo từng Object
- UI giao dịch
- UI task tracking
- Tất cả cập nhật tự động thông qua Observer pattern hoặc Unity Events.

3.2.6 Tích hợp — Cách hệ thống vận hành

Ví dụ 1: Crafting

1. Player mở Object → Crafting UI load các recipe thuộc object.
2. Khi chọn recipe:
   - CraftingManager kiểm tra Inventory.
3. Nếu đủ nguyên liệu → output item được tạo.
4. TransactionManager lưu giao dịch (nếu game có sử dụng).

Ví dụ 2: Task

1. Player nhặt item
2. InventoryManager cập nhật số lượng
3. TaskManager kiểm tra các công thức Task–Item
4. Task hoàn thành nếu đủ điều kiện

3.2.7 Lý do chọn kiến trúc này

- **Tách biệt dữ liệu, logic, và UI** → dễ mở rộng.
- **ScriptableObject** tối ưu cho dữ liệu tĩnh (items, recipes, tasks, v.v.).
- **Managers** xử lý luồng công việc rõ ràng.
- Dễ chỉnh sửa tài nguyên, giao diện và logic trong Unity Editor.
- Phù hợp với cấu trúc nhiều thực thể và quan hệ phức tạp giống ERD.